Refinement Invalidation under Mutation

Deep Dive F for "Refinement and Inference Growth in SKJS"

Models revision `bafdcf7c` of `main` (2026-08-05)

Abstract

The main soundness paper proves *one-step branch soundness*: a condition is evaluated against a fixed store, and the refined environment is guaranteed only at that instant. This paper extends the model with mutation events and determines which events must invalidate an existing narrowing. It relates each implementation path to the point where mutation can break the main paper's branch-soundness argument.

Assignments to captured locals are checked against the binding's declared or widened inferred type (**L-CAPT**). Bindings whose types intentionally evolve from an unannotated `null` or `undefined` initializer are the stated exception. Two distinct read-side failures remain: a closure can observe an enclosing narrowing after an intervening mutation invalidates it (**L-STALE-READ**), and a narrowing established before a loop can survive an invalidating write on the loop's back edge (**L-LOOP-CARRIED**). Field writes and calls create corresponding invalidation requirements for field narrowings.

The paper compares four repair policies and recommends invalidating affected locals at loop entry and closure capture, restoring a field's declared type after writes, and invalidating field narrowings across calls. It states the preservation property that this combined policy must satisfy.

Contents

1 Purpose and Ledger Rows Addressed **2**

2 Minimal Recap of the Main Paper **2**

3 A Store Model with Mutation Events **3**
3.1 The three tracked shapes 4

4 What the Implementation Does at Each Mutation Site **5**
4.1 (M-Asgn) direct assignment to a local: sound flow update, with a loop-carried hole . 5
4.2 (M-Asgn) through a closure: checked, but not propagated 6
4.3 (M-Field) syntactic field write: checked, but no update 7
4.4 (M-Call): no invalidation whatsoever 7

5 Design Space **8**
5.1 Option (i): havoc-on-call 8
5.2 Option (ii): effect / purity tracking 9
5.3 Option (iii): capture-based invalidation 9
5.4 Option (iv): status quo (documented tsc-parity unsoundness) 9

5.5 Recommendation 9

6 The Re-Widening Rule for Field Writes (#598) 10

7 Status and Registry 12

1 Purpose and Ledger Rows Addressed

This paper addresses the following rows of the main paper's Assumptions and Proof Obligations table (its §8):

- **L-PURE** (VIOLATED) — "evaluating a condition does not mutate the narrowed locals", and, by extension, the unstated companion assumption that the store does not change between the condition and the *uses* of the narrowed environment. Tracked as issues #597 and #599.
- **L-MUT** (ASSUMED, with a tracked instance) — field writes are governed by set-types; the tracked instance is issue #598: a field reassignment inside a refined branch does not re-widen the narrowed field.
- **L-CAPT** (PROVED here) — assignments to captured locals are checked against the binding's declared type (Section 4.2).
- **L-STALE-READ** (VIOLATED, owned here) — closure bodies read enclosing narrowings that are stale at invocation (Finding 6).
- **L-LOCAL** (PROVED) — the module-level half by the checker's rejection of mutable module-level bindings plus the narrowing primitive's early return; the captured-local half, which the main paper says "sharpens L-PURE", is made precise here (Finding 6).

The deliverable is a *specified* invalidation policy (Policy 7) with a soundness statement (Lemma 8), plus the exact ledger-row edits the main paper should adopt (Section 7). L-PURE remains VIOLATED until the policy lands; this paper turns it from an open design question into a fix with a stated lemma.

2 Minimal Recap of the Main Paper

We reuse the main paper's notation without redefining it: values Val with runtime tags, object values ObjVal with the snapshot interface $(\mathsf{props}, \mathsf{get}, \dots)$, type denotations $[\![\cdot]\!]_\rho$ indexed by the substitution state ρ, environments Γ mapping locals to `TType`, stores σ mapping locals to values, satisfaction $\sigma \models_\rho \Gamma$, the evaluation relations $\sigma \models_b e$, and branch soundness for rules $\Gamma \vdash e \overset{b}{\rightsquigarrow} \Gamma'$ (main paper, Definitions "Environments and Stores" through "Branch Soundness"). Three recap points are necessary here:

1. **One-step scoping.** The main paper's branch-soundness theorems say: if $\sigma \models_\rho \Gamma$ and $\sigma \models_b e$ then $\sigma \models_{\rho'} \Gamma'$ — for the *same* σ. No claim is made about later program points; assignment, loops, and joins are covered only by merge soundness (its Lemma "Merge Soundness"). The gap between "Γ' holds at branch entry" and "Γ' holds at each use inside the branch" is exactly the subject of this paper.

2. **Snapshot object semantics.** Membership $o \in [\![\mathsf{object}(B)]\!]$ constrains the properties of o "at the moment a condition is evaluated" (main paper, Definition "Object Interface"), via get-types only; set-types govern which writes the checker accepts (**L-MUT**) and do not participate in membership.

3. **The refinement primitive writes to the scope.** All narrowing lands in the flow-sensitive scope by `setLocalFilterMap`/`scope.set`; field narrowing (R-EqField, R-TruthyField) rewrites the field's `get` inside the local's object type via `narrowField` → `updateField` → `TField::withGet`, leaving `set` untouched.

impl: `TypeCheckRefine.sk / setLocalFilterMap, narrowField; TType.sk / TField::withGet`

The scope chain. Because the invalidation question is about *where* flow information lives, we fix the implementation's scope discipline once. A `Scope` is a mutable frame with an `up` pointer, per-frame `isLoop` and `isFunction` flags (the latter marks lambda bodies), declared names (`locals`), and flow entries (`refs`). A flow entry `LocalRef` carries the current (narrowed) `type` *and* the binding's `declaredType` and `isAnnotated` bit, preserved across narrowing writes (`TypeCheckScope.sk:14-19`). Lookup (`maybeGetRefHelper`, `TypeCheckScope.sk:62-75`) walks the `up` chain and records whether it crossed a loop frame; `isThroughFunction` (`TypeCheckScope.sk:115-132`) separately detects captures by walking the declaration chain. The central asymmetry: `Scope::set` (`TypeCheckScope.sk:165-204`) *always writes into the current frame*, even when the binding was found higher up — flow updates shadow rather than mutate the defining frame — after one of three checks on the written type: an *annotated* binding's writes are checked against the declared type wherever they occur; an un-annotated binding's write that escapes the current function is checked against the widened inferred type, except for evolving bindings (bare `let x = null/undefined`, recognized by `getSingleVar`); otherwise a write through a loop frame is checked against the loop-entry type. Structured constructs reconcile shadow frames explicitly: `if/else` via `merge` (pointwise union) or `popReplace` when one side terminates; `switch` clauses via `mergeN` (with a no-match path exactly when no `default` clause is present *and* the finite case set does not exhaust the scrutinee) and `uniteFallenInto` for fallthrough; loop frames via `popMerge` plus `break`/`continue` snapshots (`snapshotTo`/`uniteSnapshot`); `try` constructs via a write-log — every runtime write is recorded (`TCEnv::writeVar` → `recordTryWrite`) and a `catch`/`finally` is entered with the try's logged writes unioned in (`uniteTryWrites`), with the log routed along every escape channel (`propagateTryWrites`, `foldFinallyIntoSnapshot`). Closure bodies are the exception and the subject of Section 4.

impl: `TypeCheckScope.sk / LocalRef, Scope::set, maybeGetRefHelper, isThroughFunction, merge, mergeN, popMerge, popReplace, snapshotTo, uniteTryWrites; TypeCheckEnv.sk / writeVar, withTryWriteLog; TypeCheckStmt.sk / If, While, For, Switch, Try arms`

3 A Store Model with Mutation Events

The main paper's store σ maps locals to values and is fixed. To talk about invalidation we split it and let it step.

Definition 1 (Two-Level Store)**.** *A store is a pair $\sigma = (\sigma_L, \sigma_H)$ where $\sigma_L : \mathsf{Var} \rightharpoonup \mathsf{Val}$ maps locals to values and σ_H is a heap giving each object identity its current property map; the snapshot interface $\mathsf{get}(o, p)$ of the main paper's the Object Interface definition is read as $\mathsf{get}_{\sigma_H}(o, p)$. Satisfaction $\sigma \models_\rho \Gamma$ means $\sigma_L(x) \in [\![\Gamma(x)]\!]_\rho^{\sigma_H}$ for all $x \in \mathrm{dom}(\Gamma)$, where object-type membership consults σ_H. All results of the main paper hold verbatim for a fixed (σ_L, σ_H).*

Definition 2 (Mutation Events)**.** *Between two checker-visible program points the store may step* $\sigma \xrightarrow{\mu} \sigma'$ *by one of:*

- ***(M-Asgn)*** *direct assignment* $x := v$*: replaces* $\sigma_L(x)$*;* σ_H *unchanged.*
- ***(M-Field)*** *field write* $o.f := v$*: replaces one property in* σ_H*;* σ_L *unchanged. Reachable from the checked function as* `x.f = e`*, or from* any *called code holding an alias of* o*. If* f *is accessor-backed, the event is instead "run the setter", which may perform arbitrary further events.*
- ***(M-Call)*** *call/new/await boundary: runs foreign code, which may perform any sequence of (M-Field) events on heap objects reachable from its arguments, from globals, or from closures it can reach, and any (M-Asgn) events on locals* captured by such closures*. Locals not captured by any closure are unreachable: (M-Call) cannot write them.*

JS is single-threaded per agent, so between two adjacent checker-visible points with no intervening call and no intervening syntactic write, the store is genuinely fixed; this is what makes the main paper's one-step theorems non-vacuous.[1]

Definition 3 (Preservation Discipline)**.** *The checker tracks a current environment* Γ_t *at each program point* t*. Call an environment-update function* U_μ *(what the checker does to* Γ *when it sees, or must conservatively assume, event* μ*)* adequate *for* μ *if* $\sigma \models_\rho \Gamma$ *and* $\sigma \xrightarrow{\mu} \sigma'$ *imply* $\sigma' \models_\rho U_\mu(\Gamma)$*. Flow-sensitive checking is sound iff every event that can occur between two points is matched by an adequate update on that path. The main paper's branch-soundness results supply the* U *for condition evaluation (no event,* $\sigma' = \sigma$*); this paper supplies — or exhibits the absence of — the* U*'s for (M-Asgn), (M-Field), (M-Call).*

3.1 The three tracked shapes

Each shape below is pinned by `tests/expect-error/unsound-refinement.ts` (cases 1–3); all three compile without error today, re-confirmed by running the checker at the pinned revision.

Shape (a) — #597: side-effecting call between test and use.

```
declare function sideEffect(): void;
function f(x: { f: number | null }): void {
  if (x.f !== null) {
    sideEffect();          // (M-Call): may write x.f through an alias
    const n: number = x.f; // accepted; unsound
  }
}
```

R-EqField narrows the object branch's field to f.`get` = `number` at branch entry — sound there. The call is an (M-Call) event: x's object is reachable from the caller of f (it is a parameter, hence aliased), so σ_H may step. The checker's update for the call is the identity, which is *not* adequate for (M-Call): the failing hypothesis is the fixity of σ_H — precisely assumption **L-PURE** read as "the snapshot taken by the snapshot Object Interface is still current at the use". No lemma of the main paper is wrong; its scope boundary ("one-step") is real.

[1] In a multi-agent (shared-memory) setting even that fails for `SharedArrayBuffer`-backed state; SKJS, like tsc, ignores this, and so do we.

Shape (b) — #599: captured local mutated via an invoked lambda.

```
function f(): void {
  let x: string | null = "hello";
  if (x !== null) {
    const reset = () => { x = null; };
    reset();               // (M-Call) performing (M-Asgn) on captured x
    const s: string = x;   // accepted; unsound
  }
}
```

Here the mutation is not foreign: the checker *typechecks the assignment* `x = null` while checking the lambda body. The event is visible, yet the update applied to the branch environment is the identity — Section 4 shows the assignment lands in the lambda's scope frame, which is discarded. The failing hypothesis is again **L-PURE** at the `reset()` boundary; but unlike shape (a), an exact (not conservative) update was available and thrown away.

Shape (c) — #598: field reassignment inside a refined branch.

```
function f(x: { f: number | null }): void {
  if (x.f !== null) {
    x.f = null;            // (M-Field), checked against set-type: passes
    const n: number = x.f; // accepted; unsound (tsc rejects, TS2322)
  }
}
```

A pure (M-Field) event with no call and no alias needed: the write is syntactic and checked — against the field's set-type (`number | null`, untouched by narrowing), so it passes — but the update applied to $\Gamma(x)$ is the identity, leaving the stale $f.\mathsf{get} = \texttt{number}$. The failing hypothesis is the **L-MUT** note's "membership semantics (get-only) is sound for the snapshot reads refinement performs": after the write the snapshot is stale, so $\sigma'(x) \notin [\![\Gamma(x)]\!]^{\sigma'_H}$. This shape is *not* tsc-parity: tsc's assignment narrowing rewrites the declared flow type of `x.f` to `null` and rejects the next line.
impl: `TypeCheckRefine.sk / evalFieldEqualityType (withGet); TType.sk / TFields::resolveFieldType (isLValue reads set)`

4 What the Implementation Does at Each Mutation Site

Established by code inspection at `bafdcf7c` and, where marked *(probed)*, by running the checker (`make check-file-in-docker`) on minimal programs.

4.1 (M-Asgn) direct assignment to a local: sound flow update, with a loop-carried hole

`assign`'s `Var`-LHS arm types the RHS and calls `tcenv.writeVar` — the runtime-write wrapper that records the value for any enclosing `try`'s write-log and then calls `scope.set` (`TypeCheckExpr.sk:3749-3770`, `TypeCheckEnv.sk:274-282`): the local's flow entry is *replaced* by the RHS type in the current frame. This is the adequate update $U_{\text{M-Asgn}}(\Gamma) = \Gamma[x \mapsto \tau_{\text{RHS}}]$ ($\sigma'_L(x) = v \in [\![\tau_{\text{RHS}}]\!]$ by **L-TC**). In particular a narrowed type is correctly clobbered: `if (x !== null) { x = null; const s: string = x; }` is rejected *(probed)*. `x++`/`x--` on a variable is routed through `Scope::set` like `x =`

`x + 1` (`TypeCheckExpr.sk:3097-3115`) — though directly, bypassing `writeVar`, so an enclosing try's write-log does not see it (a benign gap for types, since the operand is already checked to be `number`, but a gap in the log's completeness claim).

Finding 4 (Loop-carried narrowings are not invalidated)**.** (probed) *`Scope::set`'s checks are mutually exclusive in order annotation → capture → loop (`TypeCheckScope.sk:178-197`): for an* annotated *binding, a write through a loop frame is checked only against the declared type, and the loop-entry (possibly narrowed) type is never consulted. A narrowing established before the loop therefore survives the back edge while the store does not:*

```
declare function cond(): boolean;
function f(): void {
  let x: string | null = "hi";
  if (x !== null) {
    while (cond()) {
      const s: string = x;  // iteration 2: x is null; accepted
      x = null;             // legal against the declared type
    }
  }
}
```

compiles clean; at runtime the second iteration binds `s : string` to `null` (confirmed on Node), and tsc rejects the read (TS2322: `string | null` not assignable to `string`). No call, closure, or alias is involved — this is a pure back-edge instance of the invalidation question, and the only mutation site in Section 4 where the checker applies a wrong *update rather than a missing one: the body is checked once under the pre-loop narrowing, so iteration-2 reads are justified only if back-edge writes preserve the loop-entry type, which the declared-type check does not ensure. For* un-annotated *bindings the loop-entry check does fire and the shape is rejected. Registry row* ***L-LOOP-CARRIED****; the witness is the repro of issue report `issues/loop-carried-narrowing-stale.md`, accepted at the pin. The fix is policy layer P1 (Section 5.5).*

impl: `TypeCheckExpr.sk / assign (Var arm, line 3749), PostOp (line 3083); TypeCheckEnv.sk / writeVar; TypeCheckScope.sk / Scope::set (annotation/capture/loop arms, lines 178--197)`

4.2 (M-Asgn) through a closure: checked, but not propagated

`anonFunction` checks a lambda body in a fresh subscope created with `isFunction => true` whose `up` is the creation-site scope (`TypeCheckExpr.sk:3584`), then restores the saved scope *without any merge*: the lambda frame and every flow entry written into it are discarded. Writes to captured locals are therefore never *propagated* — but they are *checked*. This subsection proves **L-CAPT** and delimits it.

Lemma 5 (L-CAPT: captured writes are checked against the declared type)**.** *Every syntactic assignment form to a local — plain and destructuring assignment, and `++`/`--` — reaches `Scope::set` (`writeVar` call sites `TypeCheckExpr.sk:3760, 3990, 4224`; the `PostOp` route `TypeCheckExpr.sk:3102-3115`). For an* annotated *binding, `Scope::set` checks the written type against `declaredType` unconditionally (`TypeCheckScope.sk:178-183`) — the annotation is a hard constraint wherever the write occurs, including inside a lambda body. For an* un-annotated *binding whose write escapes the current function — detected by `isThroughFunction`, which walks the declaration chain and ORs the `isFunction` flags crossed (`TypeCheckScope.sk:115-132`),*

staying correct when an earlier in-lambda write already shadowed the binding — the written type is checked against the widened inferred type stored in `declaredType` *(TypeCheckScope.sk:184-194).* (Probed:) *both the annotated and the inferred-type variants of a wrong-typed captured write (*`number` *into a* `string` *binding) are rejected with TS2322 at the write.*

The evolving-binding exception (row L-CAPT-EVOLVE, violated). The inferred-type check is skipped when `declaredType` is a single unresolved type variable — a bare `let x = null`/`undefined` (`TypeCheckScope.sk:192`). Such a binding widens like TypeScript's evolving `any`, so *every* escaping write is allowed, and reads elsewhere are not informed of it — witnessed end-to-end (issue report `evolving-capture-unchecked.md`: a closure writes `"s"` into a bare `let x = null` binding and a later read binds the string at type `null`). tsc accepts the same program through the same mechanism (implicit evolving `any`), so the hole is tsc-parity — but it is a hole, and it is this paper's row **L-CAPT-EVOLVE**, which is exactly the case L-CAPT's proved statement excludes: L-CAPT quantifies over bindings with an annotation or a non-trivial initializer.

Because captured writes are checked but not propagated, the remaining closure-side unsoundness is on the *read* side:

Finding 6 (Closure bodies read narrowings that are stale at invocation — row L-STALE-READ)**.** (probed) *Because lookup walks the whole* `up` *chain (**L-LOCAL**'s captured-local half), a lambda body checked inside a refined branch* sees the narrowed type*:* `let x: string | null = "hi"; if (x !== null) { const f = () => x.length; x = null; f(); }` *compiles clean and crashes at runtime. The narrowing was valid when the closure was* created*, not when it* runs*. tsc handles this by re-widening, inside closure bodies, any* `let` *that is assigned anywhere in the function. This is the dual of shape (b): (b) is a stale the closure writes and enclosing code later reads; here enclosing code writes and the closure later reads. Confirmed with a runtime witness (a* `TypeError` *reading* `.length` *of* `null`*); the witnesses — including a field-level variant no policy layer below covers — are the two repros of issue report* `issues/stale-closure-reads-narrowing.md`*, both accepted at the pin.*

impl: `TypeCheckExpr.sk / anonFunction (line 3550: isFunction subscope, scope save/restore, no merge); TypeCheckScope.sk / Scope::set, isThroughFunction, maybeGetRefHelper`

4.3 (M-Field) syntactic field write: checked, but no update

For `x.f = e` (with `x` not a tracked growable local object), `assign` falls to its default arm: it types the LHS in lvalue mode — `dot` with `AKLValue`, which resolves the field via `resolveFieldType(.., isLValue=true)` = the field's `set`-type, intersected across members — checks $\tau_{\text{RHS}} <:$ that, and returns. *It never touches the scope entry for* `x`: there is no `scope.set` on this path, so a narrowed f.`get` survives its own overwrite (shape (c)). The narrowing machinery and the write path even share the data: narrowing rewrote f.`get` via `withGet` and left f.`set` declared, which is exactly why the write still typechecks. (The separate `assignLocalObject` path for growable local objects *does* update the scope, by intersection — a growth mechanism, not an invalidation one, and out of scope here.)

impl: `TypeCheckExpr.sk / assign (default arm, line 3806), dot (AKLValue); TType.sk / TFields::resolveFieldType`

4.4 (M-Call): no invalidation whatsoever

No call, `new`, `await`, or method invocation touches `tcenv.scope`. The complete set of scope writers in the checker is: `Scope::set` via the refinement primitives (`TypeCheckRefine.sk:94, 104,`

737), via `writeVar` (`assign`'s `Var` arm, destructuring's `assignTarget`, and `assignLocalObject` — `TypeCheckExpr.sk:3760, 3990, 4224`), the `PostOp` arm (`TypeCheckExpr.sk:3104`), and the `this`-retyping after `super(...)` (`TypeCheckExpr.sk:2298`); plus the structured merges and edge folds (`merge`/`mergeN`/`popMerge`/`popReplace`, `uniteFallenInto`, `uniteSnapshot`, `uniteTryWrites`, `seedRefs` — all in `TypeCheckScope.sk`). Call typing is absent from that list, so shapes (a) and (b) follow mechanically: the identity update is applied at every (M-Call).
impl: `TypeCheckRefine.sk; TypeCheckExpr.sk / assign, PostOp; TypeCheckEnv.sk / writeVar; TypeCheckScope.sk` (grep: `no scope writes in call paths`)

5 Design Space

Notation: for a function under check, let C be the set of locals captured by at least one closure, $W \subseteq C$ those additionally assigned *anywhere in the function* (matching tsc's discipline and the stale-read witness, whose write is in the enclosing body, not a closure; both computable syntactically in one pre-pass), and say a local is *heap-visible* if its value is an object that may be aliased — conservatively: any object-typed local other than a fresh, never-escaping literal. $\mathsf{havoc}_S(\Gamma)$ re-widens the entries of S: whole-variable narrowings back to the binding's declared type, and field-level narrowings by restoring each narrowed $f.\mathsf{get}$ (mechanisms in Section 6).

5.1 Option (i): havoc-on-call

At every (M-Call) boundary apply $\mathsf{havoc}_{W \cup \{\text{field narrowings of heap-visible locals}\}}$. *Soundness:* adequate for (M-Call) by construction — the havocked entries over-approximate every value the event can install (captured writes are bounded by the declared type, Lemma 5, with evolving bindings havocked to their widened union instead; field writes by set-types via **L-MUT**), and unhavocked entries are unreachable by the event (Definition 2: uncaptured locals cannot be written by a call). *Precision cost:* the breaking shape is "narrow once, then several statements each containing a call and a later use of the narrowing". Spot-checks against the repository corpus at the pinned revision:

- `examples/src/type_refinement_examples.ts` (`processResponse`): `response.kind === "success"` narrowing used once, *inside the argument* of the following `console.log` — argument evaluation precedes the call, so a call-boundary havoc does not break it. Survives.
- `examples/src/deck.ts` (display loop): `card` narrowed by `!== undefined`, used only in the argument of one `console.log`. Survives.
- `examples/src/runtime_demo.ts` / `builtins_documentation.ts`: `aliceAge !== undefined` then `aliceAge.toString()` inside one call argument. Survives — but note `aliceAge` is not captured and not object-typed, so option (i) as scoped above would not havoc it anyway; only a cruder "havoc everything" would, and even that survives these single-use sites.
- Breaking witness (crude variant): any `if (x) { log(x.a); log(x.b); }` with `x` a narrowed *field path or captured local* — the first `log` havocs, the second use fails. `examples/src/try_catch.ts` (uses `e.message` after an earlier statement in the branch) is the nearest real instance; it survives the scoped option (i) only because the intervening statements contain no call.

The corpus tolerates the *scoped* havoc well: narrowed values are overwhelmingly consumed before or inside the first following call. The cost concentrates on ADT-style field narrowings used across several statements with interleaved calls — common in real code, rarer in this repo.

5.2 Option (ii): effect / purity tracking

Havoc only at calls whose callee may write the relevant location. SKJS already builds a call graph during checking (`CGNode`, `recordCalls`), so the skeleton exists; what is missing is (1) a per-function summary "writes captured local x" / "writes some field f through some alias", computed to a fixpoint over the call graph including indirect calls through function-typed values — which in practice degrades to "any unknown callee may write anything", (2) summaries for `declare`d ambient functions, which have no bodies — so either effect annotations are added to the language (a major design commitment tsc has repeatedly declined) or ambient calls havoc everything, and (3) soundness of the summary under first-class functions, requiring the call graph itself to be sound, which it is not required to be today (it exists for diagnostics/codegen ordering). *Verdict:* strictly dominates (i) in precision when it fires, but its sound version collapses to (i) at every ambient or indirect call — the overwhelmingly common case (`console.log` is ambient). Not recommendable as the primary mechanism; viable later as a precision overlay on top of (i)/(iii).

impl: `Reachability.sk / CGNode; TypeCheckEnv.sk / recordCalls`

5.3 Option (iii): capture-based invalidation

Precompute W (captured-and-assigned locals). Then: (1) at every (M-Call) boundary, havoc_W; (2) inside closure bodies, do not let narrowings of W-locals flow in — resolve them at their declared type (fixing Finding 6). The prerequisite that makes "declared type" a sound havoc target — captured writes bounded by the declared type — is established checker behavior (Lemma 5); for the evolving-binding exception the havoc target is the widened union of the binding's writes. This is sound for *whole-local* narrowings: an (M-Call) can write exactly the W-locals (Definition 2), and each write is bounded by the declared type. It deliberately does *not* address field narrowings (shape (a)) — the heap is aliased regardless of capture. tsc implements a cousin of this policy in restricted positions (narrowings of `var`s and of `let`s assigned in nested functions are dropped inside closure bodies, and loop-carried narrowings are reset), but performs no call-boundary invalidation at all. *Precision cost:* negligible in the corpus — no repository example narrows a captured-and-assigned local and then calls; the pinned #599 reproducer is the canonical instance and is exactly what should break.

5.4 Option (iv): status quo (documented tsc-parity unsoundness)

Keep everything, document L-PURE as violated (done), pin the reproducers (done). This is genuine tsc parity for shapes (a) and (b) — tsc accepts both — but *not* for shape (c), Finding 4, or Finding 6, all of which tsc rejects. Status quo is therefore not a coherent resting point: those are SKJS bugs that tsc rejects, not compatibility choices.

5.5 Recommendation

Policy 7 (Adopted invalidation policy)**.** *Layered, in decreasing order of urgency:*

1. ***P1 (bug fix, Finding 4):*** *invalidate loop-carried narrowings. Either extend `Scope::set`'s loop-write check to annotated bindings — a back-edge write is checked against the loop-entry (possibly narrowed) type* in addition to *the declared type — or, matching tsc, re-widen at loop entry the narrowings of every binding the loop body assigns.*

2. ***P2 (#598):*** *re-widen the narrowed field getter at every syntactic field write (rule in Section 6).*

3. ***P3 (#599 + Finding 6):*** *option (iii) capture-based invalidation for W-locals: havoc at calls, block narrowing inflow to closure bodies.*

4. ***P4 (#597):*** *option (i) scoped to field-level narrowings of heap-visible locals, applied at (M-Call) boundaries. This is the only layer with a real precision cost; if product priorities delay it, the deviation is exactly tsc's, and L-PURE's ledger row shrinks to this one named residue rather than three shapes.*

Lemma 8 (Preservation across mutation events — replaces one-step scoping at mutation sites)**.** *Assume the main paper's axioms (**L-UNION**, **L-INTER**, **L-INTER-BOT**, **L-EXP**, **L-MONO**, **L-TC**, **L-SUB**) and the well-formedness condition $[\![f.\mathsf{get}]\!] \supseteq$ the values a read of f can yield after an accepted write (Obligation 10). Under Policy 7 (P1–P4), for every checker path point t with environment Γ_t and every event $\sigma \xrightarrow{\mu} \sigma'$ occurring between t and the next point t': $\sigma \models_\rho \Gamma_t$ implies $\sigma' \models_\rho \Gamma_{t'}$, where $\Gamma_{t'}$ is the environment produced by the policy's update for μ (Definition 3): $U_{\text{M-Asgn}} = [x \mapsto \tau_{\text{RHS}}]$ with the L-CAPT check (Lemma 5) bounding captured writes and the P1 loop-entry check bounding back-edge writes; $U_{\text{M-Field}}$ the P2 re-widening; $U_{\text{M-Call}} = \mathsf{havoc}_W \circ \mathsf{havoc}_{\text{fields}}$ (P3, P4). Consequently branch soundness extends from branch entry to every use in the branch, by induction over the events on the path.*

Proof sketch. Case (M-Asgn): **L-TC** for the RHS; for captured writes Lemma 5 bounds the written value by the declared type, which is what havoc later restores (evolving bindings, outside the lemma's scope, havoc to the widened union of their writes); for a back-edge write the P1 check bounds it by the loop-entry type, which is what iteration-2 reads assume. Case (M-Field): Lemma 9. Case (M-Call): the event can write only W-locals and the heap (Definition 2); havoc_W restores declared types, sound by the Lemma 5 bound; $\mathsf{havoc}_{\text{fields}}$ restores get-types whose adequacy is Obligation 10; entries outside both havoc sets denote locations the event cannot write, so their satisfaction transports unchanged. □

6 The Re-Widening Rule for Field Writes (#598)

Rule W-Field. On `x.f = e` with RHS type τ_{RHS} (after the existing $\tau_{\text{RHS}} <: \bigcap f.\mathsf{set}$ check), update `x`'s scope entry by the memberwise primitive $\mathsf{refine}^{g,\mathsf{None}}$ with mapper: non-object members unchanged; each object branch β with a declared field f becomes

$$\beta[f.\mathsf{get} \mapsto \mathsf{union}(f.\mathsf{get},\ \tau_{\text{RHS}}),\quad f.\mathsf{opt} \mapsto f.\mathsf{opt}]$$

and branches without f (or reached only via an index) are unchanged. Optionality is *kept*: clearing it would be sound for data fields (the write makes f present) but unsound for a branch whose write went through a setter-less index — keep the conservative reading.

This reuses the exact machinery of `narrowField`: the same `setLocalFilterMap` walk, the same `updateField`/`withGet` write-back — with union where narrowing used inter/rmc, and with the branch-dropping arm removed (a write never refutes a branch). No new representation is needed.

impl: `TypeCheckRefine.sk / narrowField, setLocalFilterMap (mechanism to reuse); TypeCheckExpr.sk / assign (default arm | the missing scope.set site)`

Lemma 9 (W-Field is adequate for (M-Field))**.** *If $\sigma \models_\rho \Gamma$ and $\sigma \xrightarrow{o.f:=v} \sigma'$ with $v \in [\![\tau_{\text{RHS}}]\!]$ and $o = \sigma_L(x)$, then $\sigma' \models_\rho U(\Gamma)$ for the W-Field update U — provided f is data-backed in o*

and *no other local's recorded type views the same object through a wider* set*-type for* f *(cross-alias get/set coherence — registry row* ***L-MUT-ALIAS****). The latter hypothesis fails at the pin: the alias* `const y: {f: number} = x` is admitted *after narrowing and the legal write* `o.f := null` *through* `x` *is unrecorded on* y *(mechanism, executed witness, and tsc comparison: issue report* `issues/narrowed-alias-stale-write.md`*). The hypothesis is strictly stronger than Obligation 10; its site-by-site analysis — including the created views that need no declared alias at all — is deep dive H's.*

Proof. Only x's entry changes and only o's f-property changed in σ_H. Let β be the object branch with $o \in [\![\beta]\!]$. If β declares f: every constraint of β except the f-value constraint is insensitive to the event; the new value v satisfies the new get-type by $v \in [\![\tau_{\mathrm{RHS}}]\!] \subseteq [\![\mathsf{union}(f.\mathsf{get}, \tau_{\mathrm{RHS}})]\!]$ (**L-UNION**). If β does not declare f but an index covers it, the index constraint on the new value holds because the accepted write checked τ_{RHS} against the resolved set-type, which for an index-covered key includes the index's set-type, and the well-formedness of Obligation 10 carries it to the get side. Entries of other locals may alias o: their branches' $f.\mathsf{get}$ must also admit v — this is *not* guaranteed by W-Field (it updates only x) and is exactly why P4 additionally havocs field narrowings of other heap-visible locals at writes-through-aliases; for the syntactic single-local write of #598, aliasing within the same function without an intervening call also requires havocking *other* locals' narrowed f — the implementation should apply W-Field to every local whose type carries a narrowed field named f, a cheap over-approximation. □

The accessor situation. At the pinned revision SKJS does not represent `get`/`set` accessor members as get/set fields at all: the parser records an `isGetSet` marker (`FGet`/`FSet`) but the type checker never consults it, so a getter `get p(): T` is typed as a *method* `p(): T` (a setter likewise, then rejected for lacking a return type). That method-typing is itself unsound in the *call* direction — `const n: number = c.p()` type-checks but throws at runtime, and tsc rejects it (TS6234); registry row ACCESSOR-METHOD (issue report `getter-typed-as-method.md`), a known limitation (tests `callgraph-unsupported/getter-calls.ts`). For re-widening the consequence is that there are no coercing-setter fields: a getter read is never a narrowable value (it is a function), and the only fields with genuinely distinct `get`/`set` are synthetic index signatures (independent get/set variables) and readonly fields (`set`$=\bot$), neither of which runs user code on a write. So the W-Field rule's union against τ_{RHS} carries no accessor residue at present; its only residue is the cross-alias one of Lemma 9.

The accessor caveat, forward-looking. *If* accessor members were represented as get/set fields, writing `o.f = e` would run a coercing setter, which could store — and a subsequent get return — any value of the *declared* get-type, which union(narrowed get, τ_{RHS}) need not contain. The sound re-widening would then be $\mathsf{union}(f.\mathsf{get}, f.\mathsf{set})$, which contains the declared get-type whenever declared-get $\subseteq$ set holds (Obligation 10); tsc's τ_{RHS} narrowing is unsound for coercing accessors in the same way. Because `TField` records only `get`/`set`/`access` and not accessor-ness, the $\mathsf{union}(f.\mathsf{get}, f.\mathsf{set})$ form would be the sound option expressible even then — and supplying accessor-ness, so the checker builds get/set accessor fields, is the same change that closes ACCESSOR-METHOD. We therefore recommend the τ_{RHS} variant now (sound today up to the cross-alias residue, and matching tsc), revisiting the accessor case if and when accessor fields are added.

Obligation 10 (Get/set coherence)**.** *For every well-formed field, any value readable from the field after an accepted write lies in* $[\![f.\mathsf{get}]\!]$ *read at the field's* declared *(un-narrowed) type. For data fields this is immediate (the read returns the written value, bounded by* $f.\mathsf{set} = f.\mathsf{get}$ *declared); for*

accessors it needs the declared-get ⊆ set check, which SKJS should enforce at declaration (tsc's TS2380 lineage). Status: OPEN*; needed by Lemma 8's field case and by exit (1) above.*

7 Status and Registry

What this paper establishes per row, and how the specified fixes change the registry when they land:

Row	**Established here / fix impact**	**Expected status after the fix**
L-PURE	Fix specified: Policy P1–P4. Lemma 8 discharges the caller-side cases (a) #597 by P4 and (b) #599 by P3. Cases it does *not* cover: the closure-side stale read is the separate row L-STALE-READ below; implicit invocations (getters/coercions/iterators) are outside the (M-Call) event model, so P3/P4 do not fire there. The back-edge instance (Finding 4, fix P1) is adjacent: no call is involved, so it sits outside L-PURE's statement — its own row is L-LOOP-CARRIED.	the caller-side cases close once P3/P4 land; the loop and event-model cases remain
L-MUT	#598 has a specified re-widening rule (W-Field, adequate by Lemma 9 for data fields); the accessor case depends on get/set coherence (Obligation 10); the cross-view case is the separate row L-MUT-ALIAS, witnessed via aliases created after narrowing, outside W-Field's reach (issue report `narrowed-alias-stale-write.md`; analyzed in deep dive H).	data fields discharged once W-Field lands; L-MUT-ALIAS needs its own fix
L-LOCAL	Module half: mutable module-level bindings are rejected outright (`toplevel let is not supported`, probed for both plain and `declare` forms), and the narrowing primitive early-returns for any name outside the local scope (`TypeCheckRefine.sk:727-730`). Captured half: lookup walks the whole chain, so narrowings flow into closure bodies — made precise as L-STALE-READ.	module half stable; captured half governed by L-STALE-READ
L-CAPT	Proved (Lemma 5): every assignment form reaches `Scope::set`, which checks annotated bindings against the declared type everywhere and un-annotated captured writes against the widened inferred type; probed at the write for both. Scoped by the evolving-binding exception (bare `let x = null/undefined`; tsc-parity).	stable; the exception is the named residue
L-STALE-READ	Owned here (Finding 6): closure bodies read enclosing narrowings valid at creation, not invocation; witnessed in local and field-level forms (issue report `stale-closure-reads-narrowing.md`). P3 blocks inflow for captured-and-assigned locals; the field-level variant is closed by no layer.	local form closes once P3 lands; field-level variant needs its own rule
L-GETSET	Introduced here (Obligation 10): prerequisite for the sound accessor variant of W-Field.	depends on the get/set coherence check

Recommended pinned tests. Extend `tests/expect-error/unsound-refinement.ts` (cases 1–3 already pin shapes (a)–(c)) with: case 4, Finding 4 (loop-carried narrowing read on iteration 2); case 5, Finding 6 (closure reads a narrowing, local reassigned, closure invoked). Each policy layer that lands flips its cases from expect-clean to expect-error, forcing the corresponding passage of this paper and of the main paper's ledger to be updated in the same change — the maintenance discipline of the main paper's Appendix B. Additionally pin the *sound* behaviors this paper leans on, so regressions become visible: direct-assignment clobbering (`x = null` after narrowing rejects the next use), the rejection of a wrong-typed captured write in both its annotated and inferred-type forms (Lemma 5), and the un-annotated loop back-edge check — all verified by probe at the pinned revision but not currently pinned by any test.

Failed attempts and negative findings, for the record. (1) A pure "havoc everything at every call" baseline was considered and rejected without implementation: spot-checks (Section 5) show it would already break single-expression uses like `aliceAge.toString() + aliceAge.toString()` only under sub-expression granularity, but its statement-level version survives the corpus surprisingly well — the reason to reject it is principle (it havocs locals a call provably cannot reach), not corpus breakage. (2) An attempt to justify clearing optionality in W-Field failed on the index-covered-write case and was withdrawn (rule keeps optionality). (3) An attempt to state Lemma 9 without the aliased-locals havoc failed — the two-locals-one-object counterexample is noted in its proof; the lemma is honest only with the "apply W-Field to every local carrying a narrowed f" clause. (4) Effect tracking (option ii) was evaluated and found to collapse to havoc at every ambient/indirect call; recorded so it is not re-proposed without effect annotations.

Registry. The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-LOCAL	Module-level bindings are never narrowed; function-level locals are, including captured ones.	PROVED
L-PURE	Evaluating a condition does not mutate the narrowed locals.	VIOLATED
L-CAPT	Assignments to captured annotated or initializer-inferred locals are checked against the binding's declared or widened inferred type.	PROVED
L-CAPT-EVOLVE	A captured write to an evolving binding (bare `let x = null/undefined`) is unchecked, and later reads keep the stale evolved type.	VIOLATED
L-STALE-READ	A closure body invoked later reads enclosing narrowings that are stale at invocation time.	VIOLATED
L-LOOP-CARRIED	A pre-loop narrowing of an annotated binding survives a back-edge write that invalidates it.	VIOLATED
L-GETSET	Get/set coherence of accessor fields.	OPEN
ACCESSOR-METHOD	Accessor (`get/set`) members must be typed as fields, not methods; today a getter is a call signature, so `c.p()` type-checks and throws.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-UNION	`union` over-approximates set union (all levels).	B	PROVED
L-INTER	When `inter` returns a non-$\bot$ result, that result contains the semantic intersection of its operands, provided every recursive call that returned $\bot$ had disjoint operands (L-INTER-BOT).	B	OPEN
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	B	VIOLATED
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	C	PROVED
L-MONO	Legal inference-state changes during refinement preserve membership in types computed earlier.	E	PROVED
L-SUB	`Clauses::isSub` acceptance implies semantic inclusion under the current state.	D	OPEN
L-TC	Checked expressions inhabit their checked types.	main	ASSUMED
L-MUT	Field writes are governed by `set`-types; get-only membership is sound for snapshot reads.	main	ASSUMED
L-MUT-ALIAS	A checked field or index write through one mutable view preserves the membership guarantees of every live alias of the same runtime object.	H	VIOLATED